Node JS

Plan

- Rappel des bonnes pratiques JavaScript
- Introduction à Node.js
- Architecture de Node.js
- Modules et gestion de dépendances
- Node.js et le Web : HTTP, Connect & Express
- L'asynchrone en détails
- Communication temps réel
- Liaison avec la persistance des données

RAPPEL DES BONNES PRATIQUES JAVASCRIPT

TYPES DE DONNÉES

ECMAScript manipule différents types de données :

- Des primitives :
  - `null`, `undefined`,
  - `Boolean` (`true` ou `false`),
  - `Number` (`100`, `1e2`, `0x64`, `0o144`, `0b1100100`),
  - `String` (`"A"`, `'B'`, `` `C` ``),
  - `BigInt`,
  - `Symbol`
- Des objets :
  - `Object`, `Function`, `Array`, `RegExp`, etc.

COERCITION

Coercition est un gros mot qui signifie que la VM JavaScript va tenter de changer automatiquement le type des données pour réaliser les opérations demandées

- Rend possible le mélange de données de types différents
- Une des plus grandes sources de confusion en JavaScript

```
[] == false //-> true
null == false //-> false
Number(true) //-> 1
```

- Pour gérer ce problème, JavaScript propose l'opérateur ===
- Comme == mais avec une vérification de type en plus

```
1 === '1' //-> false
```

OBJETS STANDARDS

- Array, Boolean, Number, String, Date, RegExp, Math, Function

```
// Notation litéral :
/* Array   */ let a = [1, 2, 3]
/* Boolean */ let b = true; let b = false;
/* Number  */ let n = 0xEA
/* String  */ let s = "Essai"

// Instanciation d'objet
let d = new Date()

// Les expression régulières
let r = new RegExp('^.{3}$')
    r = /^.{3}$/

// Math ne contient que des méthodes et propriétés statiques
Math.PI; Math.random();

// Les fonctions sont des objets un peu spéciaux (premier ordre)
let add = function fnName(a, b) {return a + b};
add.name; //-> 'fnName'
```

STRUCTURATION DE CODE : IF

- La structure if ... else

```
1 if (/* test */) {
2   /* liste d'instructions */
3 } else {
4   /* autres instructions */
5 }
```

- Opérateur ternaire

```
1 const a = /* test */ ? /* true */ : /* false */;
```

- Quelque soit le type retour du test il est évalué comme un booléen, les valeurs "falsy" sont: false, null, undefined, 0, "", NaN
- Opérateurs booléen

```
1 const a = (/* Exp. truthy */ && /* Exp. falsy */) || /* Expression */
```

STRUCTURATION DE CODE : SWITCH

```
1  switch (/* valeur */) {
2  case /* valeur 1 */:
3      /* Liste d'instructions */
4      /* Attention au fall through */
5    case /* valeur 2 */:
6      /* Liste d'instructions */
7      break;
8    case /* valeur x */:
9      /* Liste d'instructions */
10     break;
11   default:
12     /* Liste d'instructions */
13 }
```

STRUCTURATION DE CODE : WHILE

- while

```
1 while (/* test */) {
2   /* liste d'instructions */
3 }
```

- do while

```
1 do {
2   /* liste d'instructions */
3 } while (/* test */);
```

STRUCTURATION DE CODE : FOR

- for

```
1 for (let i = 0; i < 5; i++) {
2   console.log('The number is', i);
3 }
```

- for ... in ⚠️
  - Permet d'itérer sur les propriétés d'un objet
  - Permet d'itérer sur les clés d'un tableau

```
1 const object = { prop1: 1, prop2: 2 };
2
3 for (const prop in object){
4   console.log('Property', prop, '=', object[prop]);
5 }
```

- for ... of (ES2015)
  - permet d'itérer sur les valeurs d'un objet itérable (Array, Map, Set, etc.)

STRUCTURATION DE CODE : RUPTURE DE FLUX

Il existe plusieurs solutions pour changer le flux du programme

- continue : continue la boucle avec le prochain élément
- break : sort de la boucle et continue le code
- return : sort de la fonction et continue le code
- throw : sort du traitement classique et remonte un problème de fonctionnement

LES VARIABLES

- Il n'y a pas de contrainte sur la longueur
- Un identifiant JavaScript doit commencer par une lettre, $ ou _
- Les caractères qui suivent peuvent être également des chiffres
- Il ne doit pas correspondre à un mot clef réservé
- Plusieurs syntaxes pour définir des variables : var, let et const

```
1 var maVariable1;
2 let maVariable2 = 'maValeur';
3 const maVariable3, maVariable4 = 'maValeur';
```

LES VARIABLES

- Les variables sont typées dynamiquement

```
1 let a = 5; // number
2 a = 'essai'; // string
3 a = { clef: 'valeur' }; // object
```

- Utiliser une variable qui n'a pas été définie avec un mot clé var, let ou const
  - En lecture, on obtient une exception
  - En écriture :
    - en mode laxiste : on définit une variable globale
    - en mode strict : on obtient une exception
  - L'utilisation des variables globales est à éviter

```
1 console.log(b); //-> Uncaught ReferenceError: b is not defined
2 b = 5; // création de la variable globale b
3 console.log(b); //-> 5
```

LES FONCTIONS : DÉFINITION

- Une fonction est un ensemble d'instructions
  - Une fonction peut être appelée avec des arguments
  - Une fonction retourne une valeur (par défaut undefined)
- Définition d'une fonction
  - le mot-clé function
  - un nom optionnel
  - une liste de paramètres entre parenthèses (peut être vide)
  - un corps entre accolades (peut être vide)

```
1 function nomDeLaFonction (parametre1, parametre2) {
2   /* instructions */
3 }
```

LES FONCTIONS : DÉFINITION

- Nom de la fonction
  - Les noms de fonctions fonctionnent comme les noms de variables
  - Une fonction sans nom est dite anonyme
  - La propriété name d'une fonction donne son nom
- Les fonctions sont des objets
  - Il est possible de les affecter à des variables

```
1 const uneVariable = function nomDeLaFonction (parametre) {
2   /* instructions */
3 };
```

LES FONCTIONS : LA SYNTAXE FLÉCHÉ

- ES2015 introduit une nouvelle syntaxe pour les fonctions
  - On utilise => à la place de function (mais pas au même endroit)

```
1 const nomDeLaFonction = (parametre1, parametre2) => {
2   /* instructions */
3 }
```

- Principal différence avec les fonctions classique
  - this est attaché statiquement au contexte de création de la fonction
  - arguments n'existe pas (on doit utiliser le spread operator à la place)
- On l'utilise surtout pour créer des fonctions anonymes à la volé :

```
1 function nomDeLaFonction (...args) {
2   args.forEach(arg => console.log(arg))
3 }
```

LES FONCTIONS : EXEMPLES

```
1  // Fonction nommée "foo"
2  function foo () { /**/ }
3
4  // Variable pointant sur une fonction existante
5  const fooVar = foo;
6
7  // Variable pointant sur une nouvelle fonction nommée
8  const barVar = function bar () { /**/ }
9
10 // Variable pointant sur une nouvelle fonction anonyme
11 const bazVar = function () { /**/ }
12
13 // Noms des fonctions
14 foo.name === 'foo'
15 fooVar.name === 'foo'
16 barVar.name === 'bar'
17 bazVar.name === 'bazVar' // !?
```

LES FONCTIONS : ARGUMENTS

- Pour appeler une fonction il faut disposer d'une référence
- L'appel de la fonction se fait avec les parenthèses
- Il est possible de lui passer autant d'arguments que souhaité

```
1  let foo = function bar(arg1, arg2) {
2    console.log(arg1, arg2);
3  }
4
5  console.log(foo, bar); //-> ReferenceError: bar is not defined
6  foo(); //-> undefined, undefined
7  foo(1, 'exemple') //-> 1 exemple
8  foo(1, 2, 3, 4) //-> 1 2
9
10 // arguments est un mot clé retournant tous les arguments
11 foo = function() { console.log(arguments); }
12 // Alternative moderne avec l'operateur de reste
13 foo = function(...args) { console.log(args) }
14
15 foo(1, 2, 3, 4) //-> [1, 2, 3, 4]
```

LES FONCTIONS : PROGRAMMATION FONCTIONNELLE

- Il est possible de passer une variable contenant une fonction en argument à une autre fonction
- Cela permet de passer un comportement en paramètre
- C'est le point de départ de la Programmation Fonctionnelle

```
1 function pourChaque(tableau, action) {
2   for(index in tableau){
3     action(tableau[index]);
4   }
5 }
6 pourChaque([1,2,3,5,8], console.log);
```

- ⚠ La fonction est passée sans parenthèse !
- Il s'agit de la référence sur la fonction qui est passée

LES OBJETS

- JavaScript est un langage orienté objet à prototype
  - http://fr.wikipedia.org/wiki/Programmation_orientée_prototype
  - Les objets ont une notion de prototype
  - Un prototype est aussi un objet (possibilité de chaînage)
  - Un objet accède de façon transparente à son prototype
- Les objets ont des propriétés de n'importe quel type
  - Les propriétés qui ont comme valeur une fonction sont généralement appelées méthodes
  - Le nom d'une propriété est appelé clé
- Depuis ES2015, on peu créer des objets avec une syntaxe de classes

LES OBJETS : MODIFICATION DES PROPRIÉTÉS

- Il est possible d'assigner des valeurs aux propriétés
- Mais aussi d'en ajouter et d'en supprimer

```
1  const o = {}
2
3  const x = {
4    prop0: 'initial'
5  }
6
7  o.nom = 'essai'
8
9  x.prop1 = function () {
10   console.log('hello')
11 }
12
13 delete x.prop0
14
15 console.log(o) //-> { nom: 'essai' }
16 console.log(x) //-> { prop1: function }
```

LES OBJETS : THIS

- Au sein d'une fonction, on dispose du mot clé this
- Le comportement est dépendant de la manière dont la fonction est appelée
  - Hors contexte (appel simple) :
    - En mode laxiste : this = window (browser) ou global (Node)
    - En mode strict : this = undefined
  - En contexte (constructeur, objet avec syntaxe à point) : this = l'objet
- Attention à ce piège :

```
1 const dandy = {
2   name: 'Georges Abitbol',
3   say: function () {
4     console.log("L'homme le plus classe du monde est " + this.name)
5   }
6 }
7
8 setTimeout(dandy.say, 100) // L'homme le plus classe du monde est undefined
```

LES OBJETS : CONSTRUCTEURS

- Pour définir un objet (singleton)
  - Utiliser la syntaxe litérale {}

```
1 let monObjet = { property: "field1" };
```

- Utiliser une fonction comme constructeur avec le mot clé new
  - Dans le constructeur, this représente alors l'objet
  - Toute fonction peut être utilisée comme constructeur
- (ES2015) Utiliser la syntaxe des classes

LES OBJETS : CONSTRUCTEURS

```
1  function Contact () {
2    this.nom = '';
3    this.prenom = '';
4  }
5
6  Contact.prototype.toString = function () {
7    return this.prenom + ' ' + this.nom;
8  }
9
10 let c = new Contact();
```

```
1  class Contact {
2    constructor () {
3      this.nom = '';
4      this.prenom = '';
5    }
6
7    toString () {
8      return this.prenom + ' ' + this.nom;
9    }
10 }
11
12 let c = new Contact();
```

LES OBJETS : HÉRITAGE

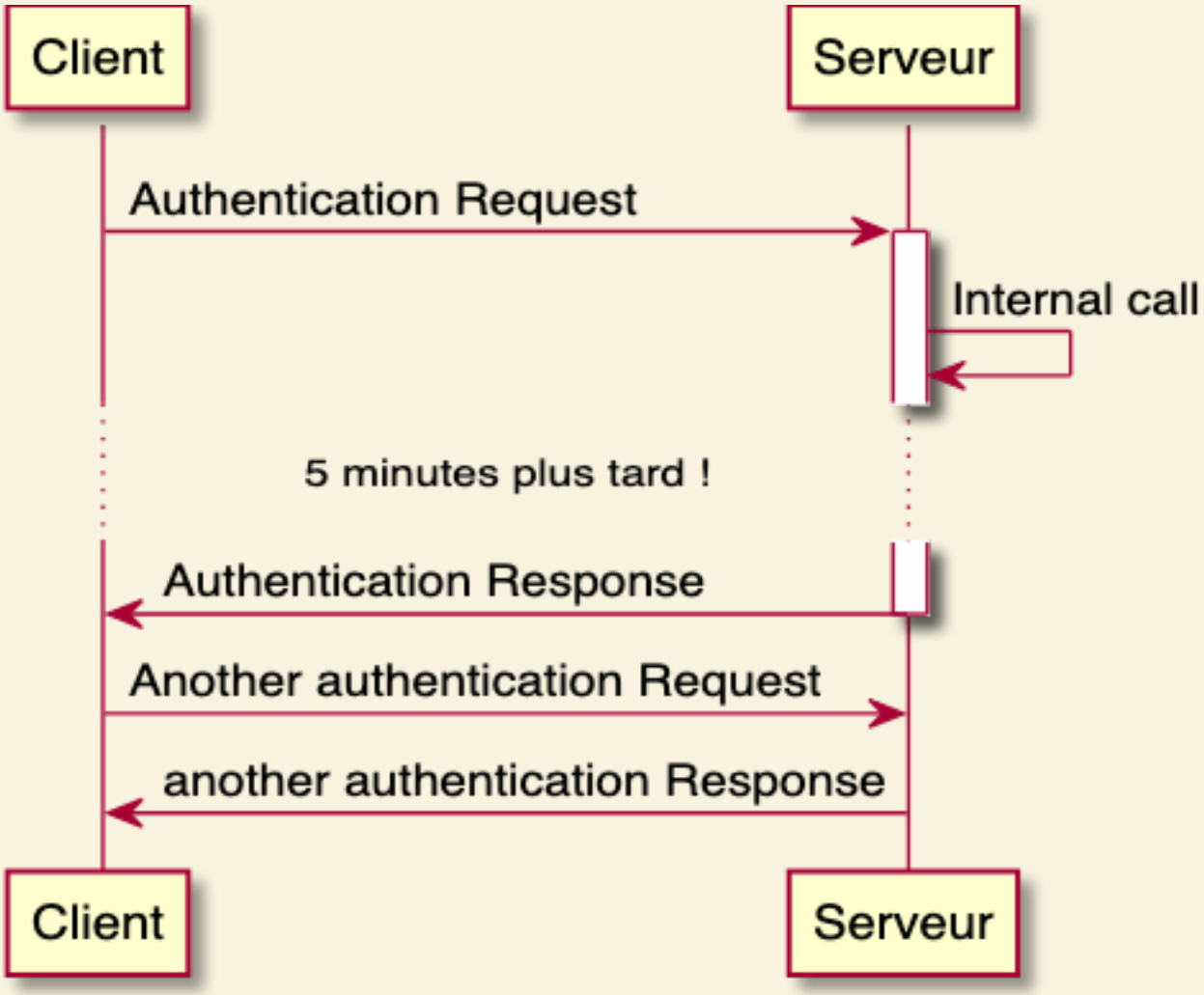
```
1  class Parent {
2      toString () {
3        return this.prenom + ' ' + this.nom;
4      }
5  }
6
7  class Contact extends Parent {
8      constructor () {
9        super();
10       this.nom = 'nom';
11       this.prenom = 'prenom';
12     }
13 }
14
15 let contact = new Contact();
16 console.log(contact.toString());
```

Architecture de Node.js

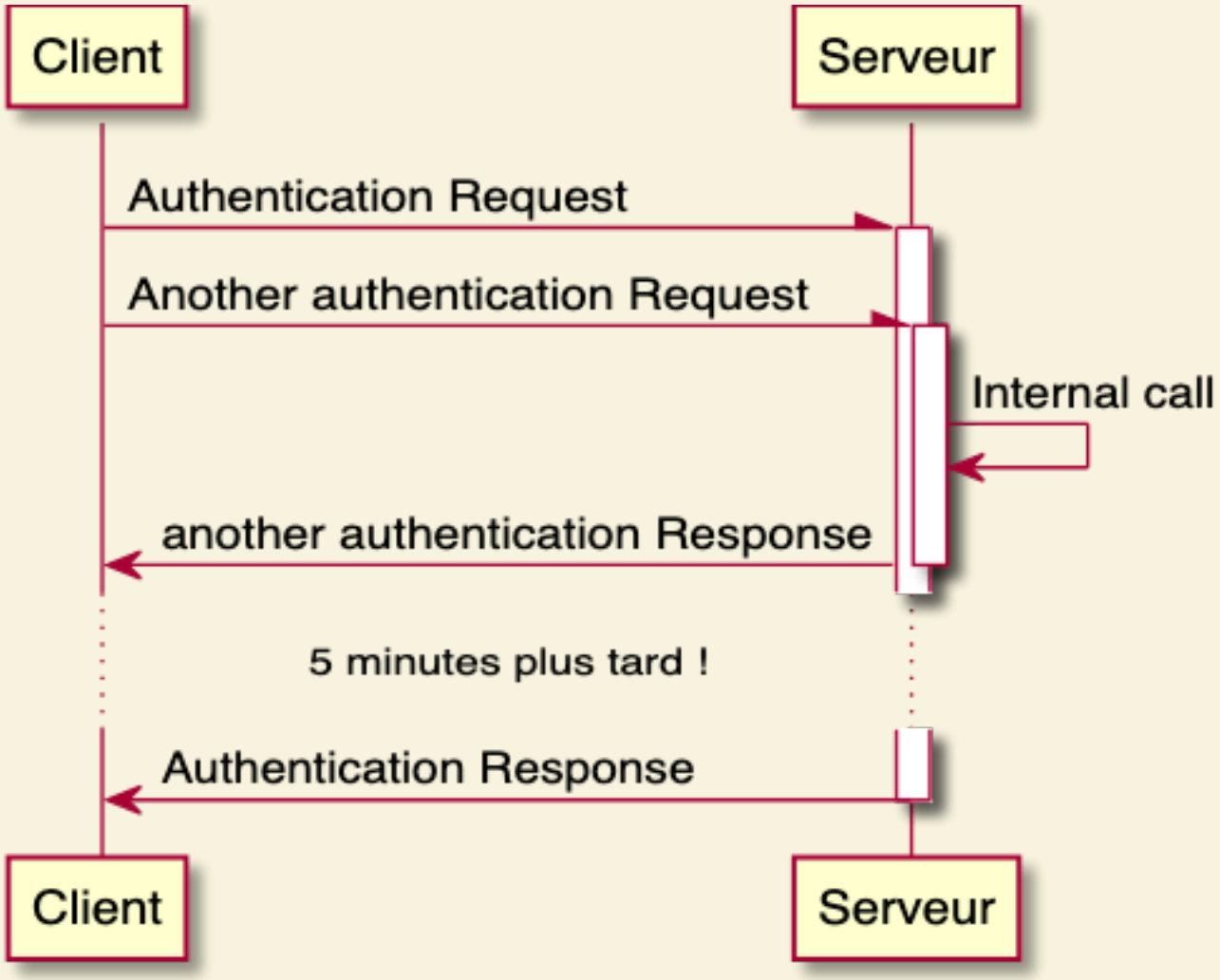
ASYNCHRONISME

- La majorité des programmes sont écrits de manière séquentielle
- La plupart des autres langages / plateformes / frameworks
  - Incitent à programmer séquentiellement
  - Multiplient les threads pour augmenter les performances
  - Ce qui crée des problèmes d'accès concurrents
  - Peuvent utiliser des I/O asynchrones mais le font rarement

ASYNCHRONISME



ASYNCHRONISME



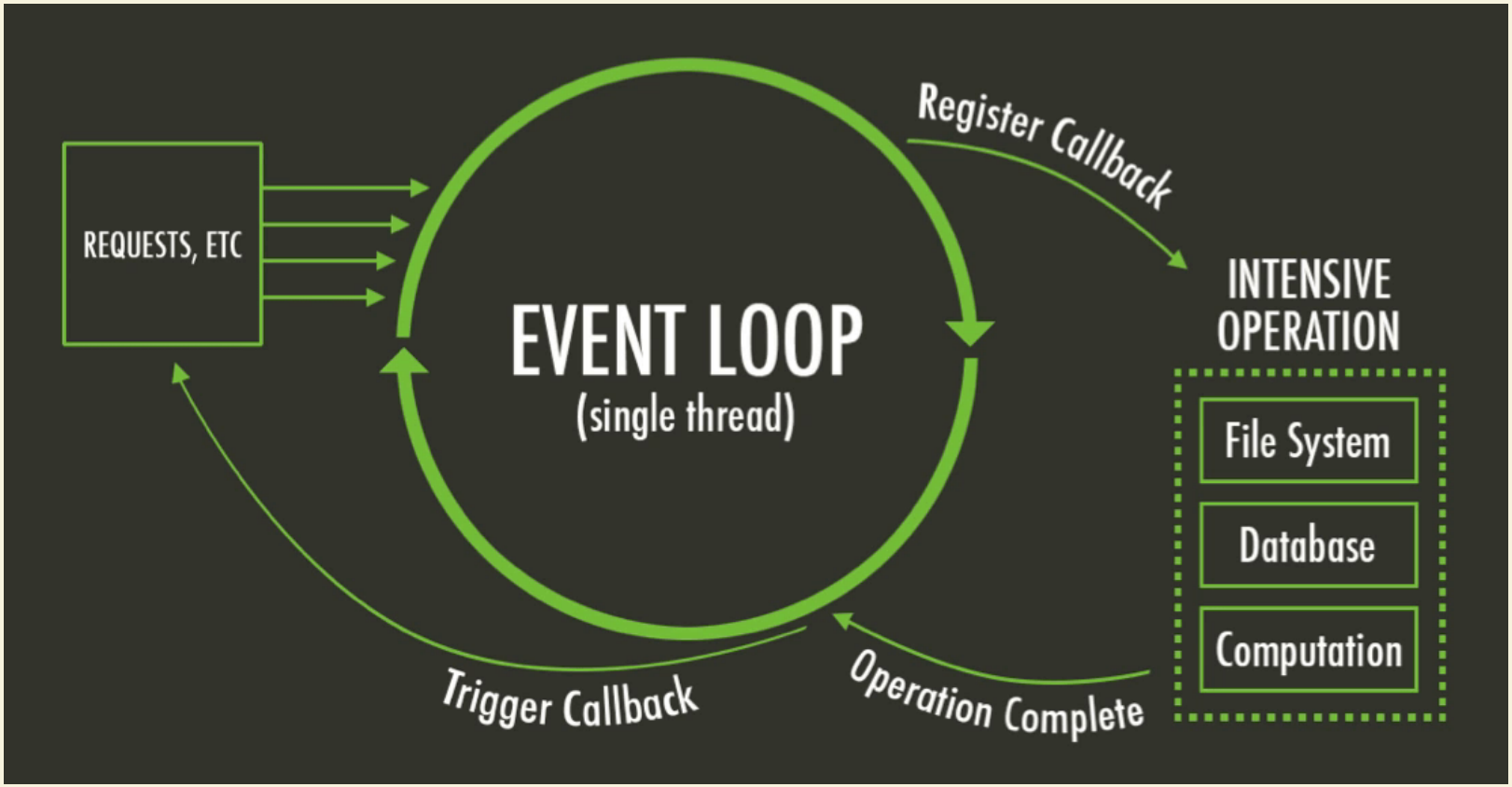
ASYNCHRONISME

- La programmation asynchrone est très impactante
- Les traitements peuvent pas *retourner la réponse*
- Il faut réaliser les enchaînements dans des callbacks
- Ces callbacks seront exécutées *plus tard*
- Le contexte de l'application aura alors peut-être changé
- Heureusement, la nature fonctionnelle du JavaScript aide beaucoup pour l'écriture du code

L'EVENT LOOP

- Le moteur JavaScript n'utilise qu'un seul thread
- Il n'est donc jamais utile de s'inquiéter des accès concurrents
- Par contre la gestion des I/O en interne utilise des threads
- Pour cela Node.js met en place un certain nombre de briques
  - Une pile FIFO des callbacks à traiter
  - Le Single Thread Event Loop qui dépile les callbacks
  - Traitements : réalisation des demandes d'I/O
  - Les traitements terminés ajoutent un élément dans la pile

L'EVENT LOOP


REQUESTS, ETC
EVENT LOOP
(single thread)
Register Callback
INTENSIVE OPERATION
File System
Database
Computation
Trigger Callback
Operation Complete

MODULES ET GESTION DES DÉPENDANCES

LES MODULES

- Chaque fichier .js dans Node.js est un module qui peut importer d'autres module et qui exporte une API utilisable par d'autres modules
- On distinguera
  - Les modules natifs qui font partie de Node.js (fs, os, path, etc. écrits en C++)
  - Les modules internes qui sont les fichiers .js qui composent l'application
  - Les modules externes qui sont des bibliothèques téléchargées (souvent des fichiers .js également)

Depuis Node.js 12.17, il existe deux API stables pour les modules : la syntaxe CommonJS et la syntaxe ECMAScript standard

LES MODULES: COMMONJS

- C'est le format historique, par défaut, des modules Node.js
- Ils utilisent une API `require()` (pour importer un autre module), `exports` (pour exporter l'API du module courant)
- Le chargement des modules CJS est synchrone

```
1 const { cwd } = require('node:process');
2 const { resolve } = require('node:path');
3
4 module.exports = (path) => resolve(cwd(), path)
```

LES MODULES: ECMASCRIPT

- Le langage JavaScript (ES2015) définie un système de module natif
- Ils utilisent une API `import` (pour importer un autre module), `export` (pour exporter l'API du module courant)
- Le chargement des ESM (ECMAScript Module) est asynchrone

```
1 import { cwd } from 'node:process';
2 import { resolve } from 'node:path';
3
4 export default (path) => resolve(cwd(), path)
```

NPM

- Node.js inclut un système de gestion des paquets : npm
- Il existe pratiquement depuis la création de Node.js
- C'est un canal important pour la diffusion des modules

NPM INSTALL

- `npm` est un outil en ligne de commande (écrit avec Node.js)
- Il permet de télécharger les modules disponibles sur npmjs.org
- Les commandes les plus courantes :
  - `install` : télécharge le module et le place dans le répertoire courant dans `./node_modules`
  - `install -g` : installation globale, le module est placé dans le répertoire d'installation de Node.js

    Permet de rendre accessible des commandes
  - `update` : met à jour un module déjà installé
  - `remove` : supprime le module du projet

NPM INIT

- `npm` gère également la description du projet
- Un module Node.js est un (ou plusieurs) script(s)
- Le fichier de configuration se nomme `package.json`
- `npm` permet également de manipuler le module courant
  - `init` : initialise un fichier `package.json`
  - `init --scope=@my-username` : permet de créer un projet scopé
- `docs <moduleName>` : ouvre la documentation du module
- `install <moduleName>` suivi de `--save` ou `--save-dev` :
  - Comme install mais référence automatiquement la dépendance dans le `package.json`

PACKAGE.JSON

- `npm` se base sur un fichier descripteur du projet
- `package.json` décrit précisément le module
- On y trouve différents types d'informations
  - Identification
    - `name` : l'identifiant du module (unique, url safe)
    - `version` : doit respecter node-semver
  - Description : `description`, `authors`, ...
  - Dépendances : `dependencies`, `devDependencies`, ...
  - Cycle de vie : scripts `main`, `test`, ...

PACKAGE.JSON DESCRIPTION

- `description` : la description détaillée
- `author` : les détails sur l'auteur du module
  - Une personne est définie avec `name`, `email`, `url`
- `contributors` : la liste des contributeurs
- `homepage` : la page web du projet
- `repository` : la référence vers le système de version
  - `type`, `url`, par défaut Git et même GitHub
  - `loginGitHub/nomDuRepo` peut suffire
- https://docs.npmjs.com/files/package.json

PACKAGE.JSON DEPENDANCES

- `dependencies`

  La liste des dépendances nécessaires à l’exécution

- `devDependencies`

  Les dépendances pour les développements (build, test...)

- `peerDependencies`

  Les dépendances nécessaires au bon fonctionnement du module (liste de compatibilité), mais pas installées lors d'un `npm install`

- `optionalDependencies` *(rare)*

  Des dépendances qui ne sont pas indispensables à l'utilisation du module, prend en compte que la récupération peut échouer

- `bundledDependencies` *(rare)*

  Des dépendances qui sont publiées et livrées avec le module

PACKAGE.JSON: VERSIONS

- Les modules doivent suivre la norme semver
  - Structure : `MAJOR.MINOR.PATCH`
  - `MAJOR` : Changements d'API incompatibles
  - `MINOR` : Ajout de fonctionnalités rétrocompatibles
  - `PATCH` : Corrections de bugs
- Pour spécifier la version d'une dépendance
  - `version` : doit être exactement cette version
  - `~`, `^` : approximativement (patch), compatible (minor/patch)
  - `major.minor.x` : `x` fait office de joker
  - Et bien d'autres : `>`, `<`, `>=`, `min-max`...

MODULES

- La modularisation dans Node.js est disponible via deux API
  - CommonJS (CJS) : http://nodejs.org/api/modules.html
  - ECMAScript (ESM) : https://nodejs.org/api/esm.html
- Principe fondateur : Chaque fichier `.js` est un module isolé
- Attention, c'est une notion en rupture avec le *côté client* !
- La modularisation apporte les commandes :
  - `require` (CJS) ou `import` (ESM) : pour charger un module
  - `exports` (CJS) ou `export` (ESM) : pour publier une API

MODULES : IMPORTATION

- Modules core : fournis par Node.js, ils sont toujours accessibles

```
1 import fs from 'fs'; // ESM
2 const fs = require('fs'); // CJS
```

- Modules npm : accessibles par leur nom s'ils se trouvent dans le répertoire `./node_modules`

```
1 import commander from 'commander'; // ESM
2 const commander = require('commander'); // CJS
```

- Modules locaux : tout fichier `.js` est un module.

```
1 import submodule from './dir/submodule.js'; // ESM
2 const submodule = require('./dir/submodule.js'); //CJS
```

MODULES : IMPORTATION

- L'opération est synchrone pour les module CommonJS et asynchrone pour les modules ECMAScript
- Les modules sont lus et interprétés dans l'ordre d'appel
- Les dépendances circulaires sont possibles et gérées
- Si le même module est chargé deux fois :
  - Le contenu n'est interprété qu'une seule fois
  - Le module est mis en cache
  - Il est rendu immédiatement s'il a déjà été chargé
  - L'identité d'un module est associé à son fichier `.js`

MODULES : EXPORTATION

- Par défaut, ce qui est dans votre module n'est pas visible
- Seul ce qui est exporté explicitement est visible

```
1 // MonModule.js
2 const str = 'ok' // Uniquement visible dans le module
3 const maMethode = () => console.log(str)
4
5 exports.maMethode = maMethode // CJS
6 module.exports = { maMethode } // CJS
7 export { maMethod } // ESM
```

```
1 // AutreModule.js
2 import monModule from './MonModule.js'; // ESM
3 const monModule = require('./MonModule'); // CJS
4
5 monModule.maMethode();
6 //-> ok
```

NODE ET LE WEB : HTTP, CONNECT & EXPRESS

NODE.JS ET LE WEB

- Node.js a été créé à l'origine pour faire du Web
- Sa focalisation sur les I/O provient de cet objectif
- Mais Node.js ne peut pas être considéré comme un serveur Web
- Node.js est une plateforme qui permet de faire un serveur Web
- Il faut programmer un serveur Web pour en avoir un
- Heureusement, c'est très simple à faire !

```
1 const http = require('http')
2
3 const server = http.createServer((req, res) => {
4   res.writeHead(200, {'Content-Type': 'text/plain'});
5   res.end('hello world');
6 })
7
8 server.listen(1234);
```

MODULE HTTP

- Node.js propose un module noyau HTTP (et aussi HTTP/2 et HTTPS)
- Il fournit un certain nombre d'APIs de bas niveau :
- Pour communiquer avec un Serveur HTTP :

```
1 const request = http.request(options, callback);
```

- Pour créer un serveur HTTP :

```
1 const server = http.createServer(requestListener).listen(port);
```

- Il supporte tout le protocole HTTP/1.1
  - Le cache : ETag, If-Modified-Since
  - Réseau: Connexions persistantes, Pipelining, Chunk Transfer Encoding...
  - Un nouveau verbe : UPGRADE

MODULE HTTP: SERVER

- Charger le module HTTP

```
1 const http = require('http')
```

- Pour créer un serveur HTTP :

```
1 const server = http.createServer(requestListener).listen(port);
```

- Définir un callback pour traiter les requêtes

```
1 server.on('request', (req, res) => {
2   res.writeHead(200, {"Content-Type": "text/plain"})
3   res.write('hello world')
4   res.end()
5 });
```

- Mettre le serveur à l'écoute d'un port

```
1 server.listen(port)
```

CONNECT

- Le module Connect
  - Framework de serveur HTTP pour Node.js
  - Ultra extensible par plugins nommés middlewares
  - Développé par TJ Holowaychuk, un grand nom des modules Node.js
  - Rarement utilisé directement, il est en dépendance de nombreuses solutions notamment Express

```
1 const app = connect()
2   .use(logger())
3   .use(serverStatic('public'))
4   .use((req, res) => res.end('hello world'));
5 http.createServer(app).listen(3000);
```

CONNECT: LES MIDDLEWARES

- static : Servir automatiquement des fichiers statiques
- favicon : Définir la favicon du site
- logger : Trace les requêtes reçues avec différents formats (standard, dev ,tiny, ...)
- query : Décoder la requête
- errorHandler : Permet de générer un traitement d'erreur particulier
- directory : Permettre la visualisation du contenu de sous dossiers
- bodyParser : Décode le body des requêtes, notamment JSON
- session : Active un système de gestion de sessions
- Une fonctionnalité n'est pas adressée : le routage

CONNECT: EXEMPLE

```
1  const http = require('http')
2  const connect = require('connect')
3  const serveIndex = require('serve-index')
4  const serveFavicon = require('serve-favicon')
5  const query = require('connect-query')
6  const errorhandler = require('errorhandler')
7  const logger = require('morgan')
8
9  const app = connect()
10   .use(logger('dev'))
11   .use(query())
12   .use(serveFavicon(__dirname + '/public/favicon.ico'))
13   .use(express.static('public'))
14   .use(serveIndex('public', {
15     hidden: true,
16     icons: true
17   }))
18   .use(errorhandler())
19   .use((req, res) => {
20     res.end(JSON.stringify(req.query))
21   })
22
23 http.createServer(app).listen(3000)
```

EXPRESS

- Le module Express
  - Toujours développé par TJ Holowaychuk
  - S'appuie sur le module Connect
  - Intègre un système de routage
  - Fournit un générateur d'application `express-generator`

```
1 const express = require('express')
2 const app = express()
3
4 app.get('/hello', (req, res) => {
5   res.send('Hello World');
6 })
7
8 app.listen(3000)
```

EXPRESS : STRUCTURE

- La structure d'un serveur Express ressemble à cela

```
1  const express = require('express')
2  const logger = require('morgan')
3  const app = express()
4
5  /* Chargement des middlewares connect */
6  app.use(express.static('./public'))
7  app.use(logger())
8
9  /* Ajout de routes */
10 app.get('/hello', (req, res) => {
11   res.send('Hello World');
12 })
13
14 app.listen(8080)
```

EXPRESS : ROUTER

- Il existe de nombreuses façons de réagir aux requêtes
- Sous forme de middleware

```
1 app.use((req, res, next) => {});
```

- Avec la méhode HTTP et l'URL

```
1 app.get('/resource', (req, res) => {});
```

- Capturer des paramètres dans l'URL

```
1 function callback (req, res) { /*req.params*/ }
2 app.get('/resource/:id', callback);
3 app.get(/^\/resource\/(\d+)$/, callback);
```

EXPRESS : LA REQUETE

- L'objet requête ressemble à celui manipulé par le module HTTP
- Express propose une API plus simple d'utilisation
  - `request.params` : les paramètres capturés dans l'URL
  - `request.query` : les paramètres 'query-string'
  - `request.body` : le body de la requête, parsé si on utilise le middleware bodyParser
  - `request.headers` : les headers de la requête
  - `request.cookies` : les cookies passés dans la requête
  - http://expressjs.com/4x/api.html#req.params

EXPRESS : LA REPONSE

- L'objet réponse ressemble à celui manipulé par le module HTTP
- Express propose une API plus simple d'utilisation
  - `res.status(code)` : spécifie un code de retour HTTP
  - `res.send(status, body)` : envoie un contenu (sans stream)
  - `res.sendFile(path)` : envoie un fichier
  - `res.redirect(url)` : renvoie une redirection HTTP
  - `res.cookie(name, value)` : positionne un cookie
  - http://expressjs.com/4x/api.html#res.status

ALLER PLUS LOIN

- Authentification avec Express
  - Connect propose une gestion automatique de sessions
  - Il utilise un cookie et conserve l'état en mémoire (stateful)
- Passport
  - Middleware Express pour gérer l'authentification
  - Très riche fonctionnellement : oAuth, SSO...
- Alternatives à Express
  - Express est incontestablement leader
  - Un concurrent important : Hapi
  - https://github.com/hapijs/hapi / https://hapijs.com

ASYNCHRONE

GESTION DE L'ASYNCHRONISME

- Deux concepts se proposent de répondre à cette complexité
  - Changer de paradigme et utiliser des *promises*
    - Différentes implémentations existent
    - L'objet `Promise` natif à V8 depuis Node.js 0.12
    - `Q` l'implémentation d'origine
  - Utiliser les Générateurs
- Ces modules sont utilisables avec Node.js ou en frontend

LES PROMESSES

- C'est un concept différent
- Petite révolution dans le monde du JavaScript
- Les promesses proposent de remplacer les callbacks
- Une promesse est un objet qui sera résolu de façon asynchrone
- Permet à un traitement asynchrone d'avoir une valeur de retour
- Il est ensuite possible d'écouter l'évènement de résolution de la promesse (avec un callback !)
- Permet de ne pas implémenter le comportement au même moment ou avant l'appel à une fonction asynchrone.

UTILISER UNE PROMESSE

- Pour utiliser une API utilisant les promesses

```
1 funcWithPromise(param1, ..., param[n])
2   .then(function success (data) {
3     /* ... */
4   });
```

```
1 const promise = funcWithPromise(param1, ..., param[n]);
2
3 promise.then(
4   function success (data) { /*... */ },
5   function error (err) { /* ... */ }
6 );
```

CHAÎNAGE DES PROMESSES

- Pour chaîner une série de traitements asynchrones

```
1  readFileWithPromise('file1.txt')
2    .then(data => {
3      // data vaut le contenu de file1
4      return readFileWithPromise('file2.txt');
5    })
6    .then(data => {
7      // data vaut le contenu de file2
8      return readFileWithPromise('file3.txt');
9    })
10   .then(data => {
11     // data vaut le contenu de file3
12   })
13   .catch(err => {
14     console.log('Erreur', err);
15   })
```

PARALLÈLE ET PROMESSES

- Pour traiter en parallèle des traitements asynchrones
- Et avoir un callback quand tous sont terminés

```
1  Promise.all([
2    readFileWithPromise('file1.txt'),
3    readFileWithPromise('file2.txt'),
4    readFileWithPromise('file3.txt')
5  ])
6    .then(results => {
7      // results est un tableau des contenus des trois fichiers
8      // (dans l'ordre des promesses)
9    })
10   .catch(err => {
11     console.log('Erreur', err);
12   })
```

ASYNC / AWAIT

- Depuis ES2017 (Node.js v7.6.0), il est possible d'avoir une écriture plus proche des appels synchrones avec les Promises grâce à la syntaxe `async/await`

```
1  function resolveAfter2Seconds(x) {
2    return new Promise(resolve => {
3      setTimeout(() => {
4        resolve(x);
5      }, 2000);
6    })
7  }
8
9  async function add (x) {
10   var a = await resolveAfter2Seconds(20);
11   var b = await resolveAfter2Seconds(30);
12   return x + a + b;
13 };
14
15 add(10).then(v => {
16   console.log(v)  // affiche 60 après 4 secondes.
17 })
```

LIAISON AVEC LA PERSISTANCE DES DONNÉES

LIAISON AVEC LA PERSISTANCE DES DONNÉES

- Node.js a besoin comme toutes les plateformes de communiquer avec les bases de données
- On distingue deux grandes catégories
  1. Les bases de données SQL : SGBDR
     - Oracle, MSSQL, PostgreSQL, MySQL, ...
  1. Les bases Not Only SQL : NoSQL
     - MongoDB, Redis, Neo4J, couchDb, ...
- Node.js est souvent trop rapidement associé au NoSQL
- Pourtant, Node.js peut très bien travailler avec des bases SQL

DRIVERS

- Comme sur les autres plateformes, il faut avoir le driver correspondant à la base de données
- On trouve sur npmjs.org un driver pour à peu près toutes les bases de données
- Il n'y a pas d'API normalisée pour Node.js
  - Chaque driver peut donc avoir sa propre API
  - Il existe des drivers qui s'imposent une API commune
  - Il existe des initiatives d'abstraction (voir plus loin)
- Les drivers ont la particularité de contenir du code natif

ORM

- Un ORM (Object-Relationnal Mapping) permet de gérer l'accès aux données à plus haut niveau
- Il existe de nombreuses solutions d'ORM pour Node.js
  - ORM (MySQL, PostgreSQL, Amazon Redshift, SQLite)
  - light-orm (MySQL, PostgreSQL, MSSQL, ...)
  - Sequelize (MySQL, PostgreSQL, SQLite, MariaDB)
  - Bookshelf/knex (Postgres, MSSQL, MySQL, MariaDB, SQLite3, Oracle, and Amazon Redshift)
  - CORMO (MySQL, MongoDB, SQLite3, PostgreSQL)
  - Model (PostgreSQL, MySQL, SQLite, Riak, MongoDB, LevelDB, In-memory, Filesystem)
  - persist (SQLite3, MySQL, PostgreSQL, Oracle)
  - streamsql (MySQL, SQLite3)

SEQUELIZE

```
1  const Sequelize = require('sequelize')
2
3  const sequelize = new Sequelize('test', 'root', 'root', {
4    dialect: 'MySQL', // ou 'SQLite', 'postgres', 'mariadb'
5    port: 3306,
6  })
7
8  sequelize
9    .authenticate()
10   .complete(err => {
11     if (err) throw err;
12     console.log('Connection has been established successfully.');
13   })
14
15 const User = sequelize.define('User', {
16   username: {
17     type: Sequelize.STRING,
18     allowNull: false
19   },
20   password: Sequelize.STRING
21 }, {})
```

SEQUELIZE: MODEL

- Sequelize permet de définir le mapping des objets
- Nécessite de faire le lien entre
  - Les propriétés de l'objet
  - La colonne de la table
- Il est possible de définir les propriétés SQL
  - Type de la colonne
  - Contraintes : `null`, `default`...
- Sequelize propose une fonctionnalité `sync`
  - Permet de créer la structure dans la base
  - Gère la création et la mise à jour de la structure

MONGOOSE

- Mongoose est un ORM pour MongoDB
- Il est très utilisé pour les applications Web Node.js
- Connection au serveur MongoDB

```
1 const mongoose = require('mongoose');
2 mongoose.connect('mongodb://localhost/test', { useMongoClient: true, });
```

- Définition du mapping des objets

```
1 const userSchema = new mongoose.Schema({
2   username: String,
3   password: String
4 });
5 const User = mongoose.model('User', userSchema);
```

MONGOOSE

- Mongoose propose de nombreuses options de mapping
- Les types de données
  - String, Number, Date, Boolean, Sous-documents, Tableau, Mixed, ObjectId, Buffer
- Il est possible d'ajouter à un mapping
  - Des méthodes d'instance, statiques, getters et setters
  - Validations et intercepteurs
  - Defaults et required
  - Indexes
  - Plugins...
- http://mongoosejs.com/docs/guide.html

MONGOOSE: API

- Créer des objets et les sauvegarder

```
1 const user = new User({ username: 'john', password: '1111' })
2 user.save((err, user) => {
3   if (err) throw err
4   console.log('user', user)
5 })
```

- Requêtes

```
1 User.find({ username: 'john' }, (err, user) => {
2   if (err) throw err
3   console.log('user', user)
4 })
5
6 User.find((err, users) => {
7   if (err) throw err
8   console.log('users', users)
9 })
```